Pandas Course Notes: Series to groupby

Oct 14, 2026 · @rohit

Roadmap

These notes follow your five notebooks in the order they were taught: Series first, then DataFrames, then groupby with custom functions. Each part names a pattern, shows the code from the notebooks, and says when to use it.

Order	Notebook	What it taught
1	session_3_pandas_1	NumPy shapes, Series indexing, boolean masks, Series.apply, 20 practice questions
2	Pandas (first)	Index alignment, missing values, DataFrame basics, loc vs iloc
3	Pandas (second)	Row filters with loc, conditional updates, bucketing, apply with axis=1, reading CSVs, a first groupby
4	Group by	Datetime parsing, groupby with agg, nunique, duplicate checks, payment events
5	Groupby - 2	Exercises 1, 2, 3 and 7, custom functions in agg, groupby.apply, shares of a group total, sampling

Datasets used, with the columns that matter:

Dataset	Size	Key columns
member_events.csv	152,057 rows, 10 columns	event_id, member_id, renewal_id, event_time, event_type, channel, outcome, amount, event_cost
members.csv	8,580 rows	member_id, join_date, plan_code, annual_fee, acquisition_channel, city, status_2025_10_09, status_2026_10_09

Dataset	Size	Key columns
renewal_opportunities.csv	11,580 rows	renewal_id, member_id, renewal_number, expiry_date, renewal_deadline, renewal_fee, offered_discount
orders.csv	one row per order	order_id, customer_id, period_id, order_date, channel_id, status, gross_amount, discount_amount, refund_amount
marketing_spend.csv	144 rows	spend_id, period_id, month_start, channel_id, purpose, amount
periods.csv	2 rows	period_id, period_label, start_date, end_date, role

The notebooks also use small hand-made frames: student marks, fruit profits, five products, a 20-person frame built with seed 42, seven travel bookings, and two city and sales toy tables.

Member events run from 2024-07-03 to 2026-10-09 and have four event types: BENEFIT_USED, REMINDER_SENT, PAYMENT_ATTEMPT and SUPPORT_CONTACT.

Part 1. Foundations: shapes and the Series

A Series is a one-dimensional array with a label on every value, and you reach it by way of lists and NumPy arrays. The shape tells you how many dimensions you are holding.

Shapes

```
import numpy as np
import pandas as pd

np.array([[10, 20, 30], [190, 180, 130]]).shape   # (2, 3)  two rows, three
columns
np.array([[1], [2], [3], [4]]).shape              # (4, 1)  one column
np.array([10, 20, 30]).shape                      # (3,)    flat, one
dimension
pd.Series([10, 20, 30, 40]).shape                 # (4,)    a Series is flat
too
```

A nested list becomes a 2D array and a flat list becomes a 1D array. The shape `(3,)` is a one-item tuple, which is why the comma is there.

Creating a Series

```
ds = pd.Series([14, 15, 18, 24, 30],
               index=['Abhinav', 'Rakesh', 'Nisha', 'Riya', 'Ashish'])
ds.index        # Index(['Abhinav', 'Rakesh', 'Nisha', 'Riya', 'Ashish'],
dtype='object')
```

A Series stores two things together: the values and the index. Without `index=`, pandas labels the rows 0, 1, 2 and so on. Every later lesson, from slicing to alignment to groupby, rests on that index.

The student-marks Series used for the rest of Part 2:

```
students    = ['Gaurav', 'Aarav', 'Kiyan', 'Ayan', 'Nirvan', 'Akshay',
'Arya', 'Kabir']
marks_maths = [90, 60, 70, 80, 60, 40, 70, 85]
ds = pd.Series(marks_maths, index=students)
```

Looking up one label returns a NumPy scalar such as `np.int64(15)`, not a Series.

Part 2. Selecting from a Series

You can select from a Series by label, by position, by a list of either, or by a True/False mask. Knowing which one you are using prevents most indexing bugs.

By label and by position

On the ages Series from Part 1:

```
ds['Rakesh']     # 15   by label, because 'Rakesh' is in the index
ds.iloc[1]       # 15   by position, the second item
ds[1]            # 15, but a FutureWarning: integer keys will soon mean
labels
```

Use `.iloc` when you mean position. The bare `ds[1]` form is being deprecated because it guesses.

Slicing

On the marks Series:

Goal	Code	Rows returned
First four students	`ds[0:4]`	Gaurav 90, Aarav 60, Kiyan 70, Ayan 80
1st, 3rd, 5th and so on	`ds[::2]`	Gaurav 90, Kiyan 70, Nirvan 60, Arya 70
3rd to 6th by position	`ds[2:6]`	Kiyan 70, Ayan 80, Nirvan 60, Akshay 40
Kiyan to Akshay by label	`ds['Kiyan':'Akshay']`	the same four rows

Position slices stop before the end number, so `2:6` gives positions 2, 3, 4 and 5. Label slices include the end label, so `'Kiyan':'Akshay'` includes Akshay. The step in `[::2]` skips every other row.

Selecting several items with a list

```
whose_marks = ['Gaurav', 'Arya', 'Nirvan']
ds[whose_marks]     # Gaurav 90, Arya 70, Nirvan 60, in the order of the list
```

The result follows the order of your list, not the order of the Series.

Boolean masks

```
lolo = [False, False, False, True, True, True, False, False]
ds[lolo]            # Ayan 80, Nirvan 60, Akshay 40
```

A mask has one True or False per row, and the True rows are kept. Typing a mask by hand does not scale, so build it from a condition:

```
ds > 70             # a True/False Series, one value per student
lolo = ds > 70
ds[lolo]            # same as ds[ds > 70]: Gaurav 90, Ayan 80, Kabir 85
```

A comparison on a Series works element by element. On plain numbers, `50 > 70` returns one False.

Combining conditions

```
lolo = (ds < 70) | (ds > 85)
ds[lolo]            # Gaurav 90, Aarav 60, Nirvan 60, Akshay 40
```

Use `|` for or, `&` for and, `~` for not, and put each comparison in parentheses. Python's `or` and `and` only work on a single True or False. They are fine for `x > 10 or y < 10` with two numbers and fail on a Series.

Part 3. apply, alignment and missing values

`Series.apply` runs your own function on every value, and arithmetic between two Series matches rows by label, not by position. Both rules explain a lot of surprising output later.

apply on a Series

```
def get_grade(marks):
    if marks > 80:
        return 'High'
    else:
        return 'Low'

get_grade(90)            # 'High'  test the function on one value first
ds.apply(get_grade)      # Gaurav High, Aarav Low, Kiyan Low, Ayan Low,
                         # Nirvan Low, Akshay Low, Arya Low, Kabir High
```

Ayan scored exactly 80, which is not above 80, so he is Low. The result keeps the original index.

The notebook also set a ratings exercise: ratings of 1 rise by 30% and ratings of 5 fall by 20%. A worked answer:

```
ratings = pd.Series([1, 2, 3, 4, 5, 1, 3, 5, 2, 4, 1, 5])

def adjust(r):
    if r == 1:
        return r * 1.3
    elif r == 5:
        return r * 0.8
    return r

ratings.apply(adjust)    # 1.3, 2.0, 3.0, 4.0, 4.0, 1.3, 3.0, 4.0, 2.0, 4.0,
1.3, 4.0
```

Arithmetic aligns on the index

```
profits = pd.Series([3000, 5000, 7000, 8000], index=['Apple', 'Guava',
'Berry', 'Orange'])
qty     = pd.Series([3, 8, 9, 4],             index=['Apple', 'Guava',
'Cherry', 'Orange'])
profits * qty
# Apple 9000.0, Berry NaN, Cherry NaN, Guava 40000.0, Orange 32000.0
```

That one output shows three rules. Pandas pairs rows by label, so Guava meets Guava even though the order differs. The result index is the union of both indexes. A label found on only one side gives NaN, and the dtype becomes float64 because NaN is a float.

With integer labels the same rule makes position irrelevant:

```
ds1 = pd.Series([77, 88, 99, 109, 81, 23])
ds2 = pd.Series([22, 24, 33, 17, 8, 90], index=[5, 8, 7, 0, 3, 2])
ds1 + ds2     # 0: 94, 1: NaN, 2: 189, 3: 117, 4: NaN, 5: 45, 7: NaN, 8: NaN
```

Label 0 pairs 77 with 17, and label 2 pairs 99 with 90. Labels 1 and 4 exist only in `ds1`, and labels 7 and 8 exist only in `ds2`.

Unequal lengths behave the same way. Adding a 6-item Series to a 4-item Series with the default index gives four sums and two NaN values:

```
ds1 = pd.Series([77, 88, 99, 109, 81, 23])
ds2 = pd.Series([77, 88, 99, 109])
ds1 + ds2     # 154, 176, 198, 218, NaN, NaN
```

Comparisons are stricter

```
ds1 == ds2    # ValueError: Can only compare identically-labeled Series
objects
```

Arithmetic fills gaps with NaN, but comparison refuses to run when the two indexes differ. Both `==` and `<` raised this error in the notebook. Make the indexes identical first, for example with `reset_index(drop=True)` on both.

Assigning a Series to a column aligns too

```
ds1 = pd.Series([77, 88, 99, 109, 81, 23])
ds2 = pd.Series([77, 88, 99, 109, 81, 23], index=[5, 8, 7, 0, 3, 2])

df = pd.DataFrame({'A': ds1, 'B': ds1})   # both columns on the default index
0..5
df['A'] = ds2                             # ds2 is labelled 5, 8, 7, 0, 3, 2
# A: 109, NaN, 23, 81, NaN, 77      B is unchanged
```

Each value lands at its own label. Labels 1 and 4 have no match in `ds2`, so they become NaN, and the labels 7 and 8 from `ds2` are dropped because the frame has no such rows.

Missing values

```
ds1 = pd.Series([77, 88, 99, 109, np.nan, np.nan, 81, 23, np.nan])
ds1.isnull()                      # True at positions 4, 5 and 8 (isna() is
identical)
ds1[ds1.isnull()]                 # shows only the missing rows
ds1[ds1.isnull()] = ds1.mean()    # the mean skips NaN: 477 / 6 = 79.5
```

All three NaN values become 79.5. To fill from another Series instead:

```
ds1 = pd.Series([77, 88, 99, np.nan, 81, np.nan])
ds2 = pd.Series([22, 24, 33, 17, 8, 90])
ds1[ds1.isnull()] = ds2[ds1.isnull()]   # position 3 becomes 17, position 5
becomes 90
ds1[ds1.isnull()] = ds2                 # same result, because assignment
aligns on labels
```

Alignment also decides the last exercise in that notebook, where the two indexes only partly overlap:

```
ds1 = pd.Series([np.nan, 14, 15, np.nan, 19], index=[2, 4, 5, 7, 8])
ds2 = pd.Series([11, 7, 4, 3, 2],             index=[1, 2, 3, 4, 5])
ds1[ds1.isna()] = ds2
# 2: 7.0, 4: 14.0, 5: 15.0, 7: NaN, 8: 19.0
```

Label 2 is filled from label 2 of `ds2`, which holds 7. Label 7 stays NaN because `ds2` has no label 7. The shorter route is `ds1.fillna(ds2)`, which matches labels the same way (this call is not in the notebook).

Part 4. DataFrames: build, inspect, select

A DataFrame is a table of Series that share one index, and you pick from it either by label with `loc` or by position with `iloc`.

Building one

```
sales    = [3000, 4000, 5000, 7000, 10000]
profits  = [233, 788, 90, 855, 900]
products = ['P1', 'P2', 'P3', 'P4', 'P5']

df = pd.DataFrame({'Sales': pd.Series(sales),
                   'Profits': pd.Series(profits),
                   'Product': pd.Series(products)})
```

Each dictionary key becomes a column. The Series are lined up on their index, so the alignment rule from Part 3 applies here too.

Picking columns

Code	Returns
`df['Sales']`	a Series
`df[['Sales', 'Product']]`	a DataFrame with two columns
`df[['Sales']]`	a DataFrame with one column
`df['Sales', 'Product']`	KeyError

The last one fails because pandas reads the tuple as a single column name, a form used for multi-level columns. Double brackets mean a list of columns. `type(df['Sales'])` is a Series and `type(df[['Sales']])` is a DataFrame.

Picking rows

```
df.iloc[0]        # the first row as a Series: Sales 3000, Profits 233,
Product P1
df.iloc[[0, 1]]   # a DataFrame with the first two rows
df.iloc[[0]]      # a DataFrame with one row, because the double brackets
keep the table shape
```

A single row comes back as a Series whose index is the column names. Its dtype is object because the columns mix numbers and text.

Inspecting

Call	Tells you
`df.index.tolist()`	the row labels, here `[0, 1, 2, 3, 4]`
`df.columns.tolist()`	the column names
`df.head(2)`, `df.tail(1)`	the first two rows, the last row
`df.dtypes`	the type of each column: Sales int64, Profits int64, Product object
`df.describe()`	count, mean, std, min, quartiles and max for the numeric columns
`df.info()`	row count, non-null count per column, dtypes and memory use

On this frame `describe()` reports a mean Sales of 5800 and a mean Profits of 573.2.

loc versus iloc

	`loc`	`iloc`
Selects by	labels	integer positions

	`loc`	`iloc`
Slice end	included	excluded
Columns given as	names	positions

```
df.loc[1:2, ['Profits', 'Product']]    # rows labelled 1 and 2, two named
columns
# Profits 788, Product P2
# Profits 90,  Product P3
```

The practice below uses a 20-person frame. The seed makes the random values repeat on every run.

```
np.random.seed(42)
data = {'ID': range(1, 21),
        'Name': ['Person_' + str(i) for i in range(1, 21)],
        'Age': np.random.randint(18, 60, size=20),
        'City': np.random.choice(['New York', 'Los Angeles', 'Chicago',
'Houston', 'Phoenix'], size=20),
        'Score': np.random.uniform(0, 100, size=20)}
df = pd.DataFrame(data)
```

Task	Code	Returns
6th to 12th row	`df.iloc[5:12]`	7 rows, Person_6 to Person_12
8th row	`df.iloc[7]`	Person_8 as a Series
7th, 9th, 13th and 18th rows	`df.iloc[[6, 8, 12, 17]]`	Person_7, 9, 13 and 18
6th row to the end	`df.iloc[5:]`	15 rows
Last two rows	`df.iloc[-2:]`	Person_19 and Person_20
First two rows	`df.iloc[0:2]` or `df.loc[0:1]`	Person_1 and Person_2
8th to 12th row, first two columns	`df.iloc[7:12, 0:2]`	ID and Name of Person_8 to 12
7th, 9th, 11th rows, columns City and Score	`df.loc[[6, 8, 10], ['City', 'Score']]`	3 rows, 2 columns

Task	Code	Returns
7th, 9th, 11th rows, last two columns	`df.iloc[[6, 8, 10], -2:]`	the same City and Score columns

The last two rows of that table were left blank in the notebook, so those answers are worked ones. Three traps appear in this practice:

- `df.loc[0:1]` equals `df.iloc[0:2]` only because the index here is 0, 1, 2 and so on. Once the index changes, the two differ.
- The notebook cell holds two lines, `df.loc[7:11, ['Name', 'Age']]` and `df.iloc[7:12, 0:2]`, and only the last line prints. They are different selections: the first returns Name and Age, the second returns ID and Name.
- The comment 'indexes 7, 9, 13, 18' was answered with positions 6, 8, 12, 17. With this default index, labels 7, 9, 13 and 18 would be `df.loc[[7, 9, 13, 18]]`, which is Person_8, 10, 14 and 19.

Part 5. Filtering rows and updating values

The workhorse pattern is `df.loc[row_mask, columns]`: a True/False condition picks the rows and a list picks the columns. The same pattern writes values when it sits on the left side of `=`.

Filtering

Here `dff` is the first five rows of the 20-person frame.

```
dff = df.head()
dff.loc[[0, 1], :]                              # rows labelled 0 and 1, all
columns
dff.loc[[True, True, False, False, False], :]   # the same rows, from a hand-
built mask
dff['Age'] > 40                                 # the mask: True, True,
False, False, False
dff.loc[dff['Age'] > 40, :]                     # Person_1 (56) and Person_2
(46)
dff.loc[dff['Age'] < 30, ['City', 'Score']]     # Person_4: New York,
1.326496
dff.loc[dff['City'] == 'New York', ['Name']]    # Person_4
```

The `:` means every column. Rows come first and columns second. To compare against a statistic of the whole column:

```
df.loc[df['Score'] < df['Score'].mean(), ['Name', 'Score']]    # 11 people
below the mean score
df.loc[df['Age'] < df['Age'].median(), ['Name', 'Score']]      # 10 people
younger than the median age
```

Updating with loc

```
df.loc[df['Score'] < 50, 'Score'] = df['Score'] + 5
# Person_3: 45.049925 becomes 50.049925.  Person_4: 1.326496 becomes
6.326496.
```

The right side is aligned by index, so each selected row receives its own score plus 5. The tempting two-step version fails quietly:

```
df[df['Score'] < 50]['Score'] = df['Score'] + 5     # SettingWithCopyWarning,
and no change
```

`df[mask]` builds a temporary copy and the write goes into that copy. In the notebook, scores under 50 had been raised once after both lines ran, not twice. The rule is one `.loc[rows, column] = value` call, never two bracket steps in a row.

Scaling a whole column needs no mask: `df['Score'] = df['Score'] * 0.8` (written in the notebook but not run). Two more traps from the notebook:

- `dff = df.head()` followed by `dff['Score'] = ds` raised SettingWithCopyWarning, because `dff` is derived from `df`. Write `dff = df.head().copy()` instead.
- Assigning a Series with a different index aligns on labels (Part 3). A Series labelled 0, 1, 5, 7, 11 gave rows 0 and 1 the scores 99.90 and 22.22, and rows 2, 3 and 4 became NaN.

The travel-bookings frame

Seven rows, used for the rest of Part 5:

```
df = pd.DataFrame({
    'booking_id': [501, 502, 503, 504, 505, 506, 507],
    'destination': ['Bali', 'Dubai', 'Goa', 'Singapore', 'Jaipur', 'Bangkok',
'Kashmir'],
    'route_type': ['International', 'International', 'Domestic',
'International', 'Domestic', 'International', 'Domestic'],
    'channel': ['App', 'Agent', 'Partner', 'App', 'Agent', 'Partner', 'App'],
    'travellers': [2, 4, 3, 1, 5, 2, 4],
    'base_pp': [38000, 42000, 14000, 52000, 9000, 36000, 16000],
    'hotel_nights': [5, 4, 3, 6, 2, 4, 5],
    'add_ons': ['meal,wifi', 'meal,insurance', 'extra_bag', 'wifi',
'meal,priority_boarding', 'insurance,extra_bag', ''],
    'promo_code': ['SAVE10', 'FAM5', 'APPONLY7', None, 'SAVE10', 'APPONLY7',
'FAM5'],
    'refundable': ['Yes', 'No', 'No', 'Yes', 'Yes', 'No', 'Yes'],
    'luggage_total_kg': [28, 70, 38, 18, 95, 55, 60],
    'booking_date': pd.to_datetime(['2026-02-01', '2026-02-10', '2026-02-12',
'2026-02-05', '2026-02-15', '2026-02-20', '2026-02-18']),
    'depart_date': pd.to_datetime(['2026-03-05', '2026-02-25', '2026-02-28',
'2026-03-20', '2026-03-02', '2026-03-01', '2026-03-10'])})
```

Raise the price for the Agent channel by 10%:

```
df.loc[df['channel'] == 'Agent', 'base_pp'] = df['base_pp'] * 1.1
```

This line raised two problems in the notebook. First, `base_pp` is int64 and receives floats, which triggers a FutureWarning about an incompatible dtype. Cast first with `df['base_pp'] = df['base_pp'].astype(float)`. Second, the cell was run twice. Agent prices went from 42000 to 46200 to 50820 and from 9000 to 9900 to 10890, a 21% rise instead of 10%, and every later number inherited it. An in-place update is not repeatable, so rebuild the frame before you re-run it.

A derived column is element-wise:

```
df['total_price'] = df['base_pp'] * df['hotel_nights'] * df['travellers']
```

Each row multiplies its own three numbers. Dubai gives 50820 x 4 x 4 = 813,120 after the double increase. The next task, left empty in the notebook, discounts SAVE10 bookings by 10%:

```
df.loc[df['promo_code'] == 'SAVE10', 'total_price'] *= 0.9
```

Rows whose promo code is `None` simply compare as False. On the notebook's totals, Bali goes from 380,000 to 342,000 and Jaipur from 108,900 to 98,010.

Bucketing: three ways to label rows

The task: High above 400,000, Medium from 200,000 to 400,000, Low below 200,000.

Way 1: loc masks, in order.

```
df['booking_bucket'] = 'High'
df.loc[(df['total_price'] <= 400000) & (df['total_price'] >= 200000),
'booking_bucket'] = 'Medium'
df.loc[df['total_price'] < 200000, 'booking_bucket'] = 'Low'
```

Set a default first and overwrite after. The parentheses are required: `df['total_price'] <= 400000 & df['total_price'] >= 200000` fails, because `&` binds tighter than the comparisons. The notebook kept that failing version as a comment.

Way 2: Series.apply with a function.

```
def make_bucket(x):
    if x > 400000:
        return 'High'
    elif x >= 200000:
        return 'Medium'
    else:
        return 'Low'

make_bucket(200000)                                    # 'Medium'
df['apply_bucket'] = df['total_price'].apply(make_bucket)
```

Both ways gave 501 Medium, 502 High, 503 Low, 504 Medium, 505 Low, 506 Medium, 507 Medium. The if/elif chain reads top down, so each branch already excludes the ones above it and no `&` is needed.

Way 3: apply with axis=1, when the label depends on several columns.

```
def apply_price_increase(row):
    if row['refundable'] == 'Yes':
        return row['total_price'] * 1.1
    else:
        return row['total_price'] * 0.95

row = df.iloc[0]
apply_price_increase(row)                                  # 418000.0  test on one
row first
df['new_price'] = df.apply(apply_price_increase, axis=1)
# 418000, 772464, 119700, 343200, 119790, 273600, 352000
```

Call	The function receives	Use when
`Series.apply(f)`	one value	the result depends on one column
`DataFrame.apply(f, axis=1)`	one whole row, as a Series	the result depends on several columns
`DataFrame.apply(f)`	one whole column	rarely needed in these notebooks

Always test the function on one value or one row (`df.iloc[0]`) before applying it to everything.

Open exercise: free luggage allowance

The notebook ends with this task: `free_luggage_kg` is 20 per traveller on International routes and 15 per traveller on Domestic routes. A worked answer:

```
df['free_luggage_kg'] = df.apply(
    lambda r: 20 * r['travellers'] if r['route_type'] == 'International' else
15 * r['travellers'],
    axis=1)
# booking 501 -> 40, 502 -> 80, 503 -> 45
```

A vectorised version avoids the row loop: `np.where(df['route_type'] == 'International', 20, 15) * df['travellers']`.

Part 6. Loading real data

Real work starts with a CSV file, so this part covers reading it, turning text into dates, and checking what the table holds before any grouping.

Reading CSV files from Drive

```
from google.colab import drive
drive.mount('/content/drive')

import os
from pathlib import Path
data_dir = Path('/content/drive/My
Drive/main_drive/iitgn/batch_2/week1/session_1/data')
os.listdir(data_dir)          # ['periods.csv', 'orders.csv',
'marketing_spend.csv']

df_periods   = pd.read_csv(data_dir / 'periods.csv')
df_marketing = pd.read_csv(data_dir / 'marketing_spend.csv')
df_orders    = pd.read_csv(data_dir / 'orders.csv')
```

`Path` objects join with `/`, so `data_dir / 'orders.csv'` builds the full path. Calling `.head()` on each table gave the first look:

- `periods` holds two fiscal years: FY2024_25 (previous, 2024-04-01 to 2025-03-31) and FY2025_26 (current, 2025-04-01 to 2026-03-31).
- `marketing_spend` has 144 rows of spend by month, channel (APP, WEB, STORE) and purpose (Acquisition, Retention).
- `orders` has one row per order, with status, gross, discount and refund amounts.

Pulling one customer shows the grain of the table:

```
df_orders[df_orders['customer_id'] == 'C000007']     # 24 orders, all
COMPLETED, across both fiscal years
```

The second set of files, from week 2, is read with dates parsed on the way in:

```
DATA_DIR = Path('/content/drive/My
Drive/main_drive/iitgn/batch_2/week2/session_1/data')
# ['members.csv', 'renewal_opportunities.csv', 'member_events.csv']

df_members  = pd.read_csv(DATA_DIR / 'members.csv', parse_dates=
['join_date'])
df_renewals = pd.read_csv(DATA_DIR / 'renewal_opportunities.csv',
                          parse_dates=['expiry_date', 'renewal_deadline'])
df_events   = pd.read_csv(DATA_DIR / 'member_events.csv', parse_dates=
['event_time'])
```

Dates

`parse_dates` turns text columns into real datetimes while the file is read. The other route converts afterwards:

```
df_events['event_time'] = pd.to_datetime(df_events['event_time'])
df_events['event_time'].isna().sum()                # 0 dates failed to
parse
df_events['event_time'].dt.strftime('%Y')           # '2024', '2025', '2026'
as text
df_events['event_time'].dt.strftime('%Y-%m-%d')     # '2024-07-03' and so
on, day only, as text
```

The columns used as group keys in the later parts come from the `.dt` accessor:

```
df_members['join_year']     = df_members['join_date'].dt.year      # integers
df_renewals['expiry_year']  = df_renewals['expiry_date'].dt.year
df_renewals['expiry_month'] = df_renewals['expiry_date'].dt.month
```

`.dt.year` returns integers and `strftime` returns text, so a group key built one way will not match a filter written the other way. Real dates also sort and compare correctly, and `min` and `max` work on them.

Profile before you aggregate

```
df_events.shape                      # (152057, 10)
df_events['event_id'].nunique()      # 152057
df_events['member_id'].nunique()     # 6380
df_events['renewal_id'].nunique()    # 11580
df_events['event_type'].unique().tolist()
# ['BENEFIT_USED', 'REMINDER_SENT', 'PAYMENT_ATTEMPT', 'SUPPORT_CONTACT']
```

The event_id count equals the row count, so each row is one event and event_id is the row key. With 6,380 members and 11,580 renewals, a member has several renewals and each renewal has several events. That tells you what to count later: rows, members or renewals.

Part 7. groupby fundamentals

A groupby splits the table into one piece per key, runs a calculation on each piece, and joins the answers into one result. Everything later in these notes is a variation on those three steps.

1. Split: rows with the same key value land in the same group.
2. Apply: a function runs on each group, such as a count, a mean or a function you wrote.
3. Combine: the answers come back as one table with one row per group.

Look inside the groups

```
df_events['event_year'] = df_events['event_time'].dt.strftime('%Y')
groups = df_events.groupby('event_year')
groups.indices
# {'2024': array([0, 1, 2, ..., 32296]),
#  '2025': array([32297, ..., 83619]),
#  '2026': array([83620, ..., 152056])}
```

`indices` maps each key to the row positions in its group. Nothing is calculated until you ask for an aggregate.

Aggregating one column

```
df_events.groupby('event_year')['member_id'].count()
# 2024    32297
# 2025    51323
# 2026    68437
```

Named aggregation

```
df_group = df_events.groupby('event_year').agg(count_rows=('member_id',
'count'))
```

Each keyword becomes a result column, and each value is a `(column, function)` pair. The group key becomes the index of the result, and what you do with it is your choice:

Ending the chain with	Result
nothing	the key stays as the index: `df_group.index.tolist()` gives `['2024', '2025', '2026']`
`.reset_index()`	the key moves back to a normal column, so the columns are event_year and count_rows
`.reset_index(drop=True)`	the key is thrown away, leaving only count_rows with index 0, 1, 2

Several results can come from one call:

```
df_events.groupby('event_year').agg(
    count_customers=('member_id', 'nunique'),
    count_events=('event_type', 'nunique')
).reset_index()
#   event_year  count_customers  count_events
#   2024        2500             4
#   2025        4000             4
#   2026        5080             4
```

The function can be a string such as `'count'`, `'nunique'`, `'mean'`, `'min'`, `'max'` or `'sum'`, or a function you wrote (Part 9).

Counting: three different questions

Function	Counts	Result in the notebook
`'count'`	non-missing values in the column, so rows when the column has no gaps	events per year: 32,297, 51,323, 68,437
`'nunique'`	distinct values	members per year: 2,500, 4,000, 5,080
`.size()`	every row in the group, missing or not	not used in the notebooks

The first number answers how many events there were. The second answers how many different members had at least one event. Confusing the two is the most common groupby mistake.

Aggregate, then label the result

```
df_orders_agg = (df_orders.groupby('customer_id')
                          .agg(total_orders=('order_id', 'count'))
                          .reset_index())
# 16,424 customers: C000001 has 9 orders, C000004 has 26

def bucket_orders(x):
    if x >= 5:
        return 'Frequent'
    else:
        return 'Regular'

df_orders_agg['order_buckets'] =
df_orders_agg['total_orders'].apply(bucket_orders)
```

The pattern is to group first and then use the labelling tools from Part 5 on the aggregated column. The aggregated table has one row per customer, so the label belongs there.

Several keys, and the duplicate check

```
df_group = df_events.groupby(['member_id', 'renewal_id', 'event_time']).agg(
    count_rows=('event_id', 'count')).reset_index()

(df_group['count_rows'] > 1).sum()          # 19
df_group.loc[df_group['count_rows'] > 1]    # the 19 groups
```

Grouping by several columns creates one group per distinct combination. If member, renewal and timestamp should identify a single event, any count above 1 is a suspect duplicate. The notebook found 19 such groups, each with exactly two rows. One example is MP000056 on RN003199 at 2025-09-04 19:30. The pattern is to group by what should be unique, count, and keep counts above 1.

Partial years

Events run from 2024-07-03 to 2026-10-09, so 2024 and 2026 are partial calendar years. The yearly counts above are not like for like. Group by fiscal period or by month when you compare periods.

Part 8. groupby with filtering

Filtering can happen before the groupby, which limits which rows are counted, or after it, which limits which groups are shown. In SQL terms these are WHERE and HAVING.

Pattern	Shape of the code	Use when the question says
A. Filter rows, then group	`df.loc[mask].groupby(...).agg(...)`	'only', 'among', 'for rows where'
B. Group, then filter groups	`df.groupby(...).agg(...).loc[lambda d: d['x'] >= n]`	'only groups with at least'

Pattern A: filter rows, then group

```
cond = (df_events['event_type'] == 'PAYMENT_ATTEMPT') & (df_events['outcome']
== 'SUCCESS')
df_events.loc[cond].groupby('event_year').agg(count_renewed=('member_id',
'nunique')).reset_index()
#   event_year  count_renewed
#   2024        2000
#   2025        3200
#   2026        3824
```

This counts distinct members with at least one successful payment. Set against the 2,500, 4,000 and 5,080 members with any event in Part 7, the shares are 80.0%, 80.0% and 75.3%. Those percentages are derived from the notebook outputs, and 2026 is a partial year, so its share may not be final.

Exercises on members and renewals

Exercise 1: member profile by acquisition channel.

```
df_members.groupby('acquisition_channel').agg(
    member_count=('member_id', 'nunique'),
    average_annual_fee=('annual_fee', 'mean'),
    earliest_join=('join_date', 'min'),
    latest_join=('join_date', 'max'))
```

The notebook cell stopped at `earliest_join =` , so this completes it. `min` and `max` work directly on date columns.

Exercise 2: paid members by city. Pattern A, not run in the notebook.

```
df_members.loc[df_members['status_2026_10_09'] ==
'PAID'].groupby('city').agg(
    paid_members=('member_id', 'nunique'),
    average_annual_fee=('annual_fee', 'mean'),
    channels_present=('acquisition_channel', 'nunique'))
```

Exercise 3: members who were PAID in 2025 and EXPIRED in 2026, then only channels with at least 300. Pattern A followed by pattern B.

```
cond = (df_members['status_2025_10_09'] == 'PAID') &
(df_members['status_2026_10_09'] == 'EXPIRED')

(df_members.loc[cond]
    .groupby('acquisition_channel')
    .agg(expired_members=('member_id', 'nunique'),
         average_annual_fee=('annual_fee', 'mean'))
    .loc[lambda d: d['expired_members'] >= 300])
```

The notebook ran the first steps and got APP 619 (average fee 1794.64), STORE 246 (1852.66) and WEB 391 (1826.37). The final `.loc[lambda d: ...]` keeps APP and WEB and drops STORE. The lambda receives the aggregated table, so it can filter on the new columns.

Exercise 7: renewal records due in September.

```
cond = (df_renewals['expiry_year'].isin([2025, 2026])) &
(df_renewals['expiry_month'] == 9)

df_renewals.loc[cond].groupby('expiry_year').agg(
    members_due=('renewal_id', 'nunique'),
    distinct_members=('member_id', 'nunique'),
    first_expiry=('expiry_date', 'min'),
    last_expiry=('expiry_date', 'max'))
```

The notebook produced the first two columns: 4,000 and 4,000 for 2025, and 4,480 and 4,480 for 2026. The last two columns finish the exercise. The two counts agree in both years, so each member has one September renewal record. If `members_due` were larger than `distinct_members`, some members would have two renewal records in the same September, which means duplicate rows or more than one renewal due.

`isin([2025, 2026])` keeps rows whose value appears in the list. It is a shorter form of two `==` comparisons joined by or. The notebook numbers its exercises 1, 2, 3 and 7, so exercises 4 to 6 are not in the files.

Part 9. Custom functions inside agg

When no built-in name does the job, write a function that takes one group's column as a Series and returns one number, then hand it to `agg`. The notebook teaches this in three steps.

1. Pull out one group's column by hand.
2. Write the function and test it on that Series.
3. Pass it to `agg` as `name=('column', function)`.

```
df = pd.DataFrame({
    'City':    ['Delhi', 'Delhi', 'Jaipur', 'Jaipur', 'Mumbai', 'Mumbai'],
    'Product': ['Pen', 'Notebook', 'Pen', 'Notebook', 'Notebook', 'Pen'],
    'Sales':   [200, 500, 150, 350, 600, 400]})

def func(ds):
    return (ds > 400).sum()

ds = df.loc[df['City'] == 'Delhi', 'Sales']      # step 1: one group by hand
func(ds)                                          # step 2: 1

df.groupby('City').agg(sales_greater_than_400=('Sales', func))     # step 3
# Delhi 1, Jaipur 0, Mumbai 1
```

`ds > 400` is a True/False Series and `.sum()` counts the Trues. Mumbai's 400 is not above 400, so Mumbai counts only its 600. A second example uses the spread of a group:

```
def range_func(ds):
    return ds.max() - ds.min()

df.groupby('City').agg(range_sales=('Sales', range_func))
# Delhi 300, Jaipur 200, Mumbai 200
```

The rule is that the function must reduce the whole Series to one scalar. A function that returns a Series as long as its input is a transform job (Part 11).

Five exercises on the member data

1. Cost range per event type.

```
def cost_range(s):
    return s.max() - s.min()

df_events.groupby('event_type').agg(
    event_count=('event_id', 'count'),
    cost_range=('event_cost', cost_range))
# cost_range: BENEFIT_USED 57, PAYMENT_ATTEMPT 10, REMINDER_SENT 0,
SUPPORT_CONTACT 0
```

A range of 0 means every row of that type has the same cost. The notebook ran only the `cost_range` column, and `event_count` is the other half of the exercise.

2. Share of renewals offered a discount, by renewal number.

```
def discount_percentage(s):
    return (s > 0).sum() / len(s) * 100

df_renewals.groupby('renewal_number').agg(
    discount_percentage=('offered_discount', discount_percentage))
# 1     7.884013
# 2    10.823529
# 3    11.722222
```

The first group tested by hand gave 7.884013, which matches. Discounts become more common at later renewals. The shorter form is `(s > 0).mean() * 100`, because the mean of a True/False Series is the share of Trues.

3. Failure rate of payment attempts, by channel.

```
payments = df_events[df_events['event_type'] == 'PAYMENT_ATTEMPT']

def failure_percentage(s):
    return np.round((s == 'FAILED').sum() / len(s) * 100, 2)

payments.groupby('channel').agg(
    total_attempts=('event_id', 'count'),
    failure_percentage=('outcome', failure_percentage))
# APP    7445    39.26
# WEB    5863    23.21
```

This filters the rows first (pattern A from Part 8) and groups after. The notebook printed the attempts and the failure rates in separate cells.

4. Average of only the expensive events.

```
def average_high_cost(s):
    s = s[s > 30]
    if len(s) > 0:
        return s.mean()
    else:
        return 0

df_events.groupby('event_type').agg(
    total_events=('event_id', 'count'),
    average_high_cost=('event_cost', average_high_cost))
# BENEFIT_USED 48.035811, PAYMENT_ATTEMPT 0, REMINDER_SENT 0, SUPPORT_CONTACT
45.0
```

Inside the function, `s[s > 30]` filters that group's own Series. The guard matters because the mean of an empty Series is NaN. The notebook's first attempt returned NaN for payment costs, which never exceed 12. Returning 0 is what the exercise asked for.

5. Retry rate, by channel. A retry means a renewal ID with two or more payment attempts.

```
def count_renewal_id(s):
    return (s.value_counts() > 1).sum()

payments.groupby('channel').agg(
    payment_attempts=('event_id', 'count'),
    renewal_attempts=('renewal_id', 'nunique'),
    renewals_with_retry=('renewal_id', count_renewal_id))
#       payment_attempts  renewal_attempts  renewals_with_retry
# APP   7445              5822              1183
# WEB   5863              5296              564
```

`value_counts()` turns a group's renewal IDs into one count per ID, so `> 1` marks the retried renewals. The notebook printed these three columns in separate cells and stopped at the counts. The retry rate divides retried renewals by renewals attempted: 1183 / 5822 = 20.3% for APP and 564 / 5296 = 10.7% for WEB. Those two rates are derived from the notebook's counts. One function gives the rate directly:

```
def retry_rate(s):
    return (s.value_counts() > 1).mean() * 100
```

The mean runs over unique renewal IDs, so it is the share of IDs with two or more attempts.

Part 10. groupby.apply

Use `groupby().apply(f)` when the calculation needs more than one column, because `f` then receives the whole group as a DataFrame. Named `agg` hands the function only a single column.

Average sales of the profitable rows

The same three steps as Part 9 apply: do it for one group, wrap it in a function, then apply it to every group.

```
df = pd.DataFrame({
    'City':  ['Delhi', 'Delhi', 'Delhi', 'Jaipur', 'Jaipur', 'Jaipur'],
    'Sales': [1000, 2000, 500, 1500, 800, 1200],
    'Cost':  [700, 1400, 600, 1000, 900, 800]})

dff = df[df['City'] == 'Delhi']                          # step 1: one
group by hand
dff.loc[dff['Sales'] > dff['Cost'], 'Sales'].mean()       # 1500.0

def get_profitable_sales_mean(dff):                      # step 2: the same
code as a function
    return dff.loc[dff['Sales'] > dff['Cost'], 'Sales'].mean()

df.groupby('City').apply(get_profitable_sales_mean).reset_index(name='profita
ble_avg')   # step 3
#      City  profitable_avg
# 0   Delhi          1500.0
# 1  Jaipur          1350.0
```

Delhi's profitable rows are the 1000 and the 2000, because the 500 sale cost 600. Jaipur's are 1500 and 1200. The result of `apply` is a Series named by group, and `reset_index(name=...)` turns it into a two-column table with a column name you choose.

The same idea with a guard

The task: average salary of employees with at least three years of experience, by department.

```
df = pd.DataFrame({
    'Department': ['Sales', 'Sales', 'Sales', 'Support', 'Support',
'Support'],
    'Salary':     [40000, 55000, 65000, 30000, 45000, 50000],
    'Experience': [1, 3, 5, 2, 4, 1]})

def get_high_avg_salary(dff):
    dff = dff[dff['Experience'] >= 3]
    if len(dff) > 0:
        return dff['Salary'].mean()
    else:
        return 0

df.groupby('Department').apply(get_high_avg_salary).reset_index(name='high_av
g_salary')
# Sales 60000.0, Support 45000.0
```

Sales keeps the 55,000 and 65,000 rows. Support keeps only the 45,000 row. The `len(dff) > 0` check returns 0 for a department with nobody qualifying, instead of NaN.

Two columns at once on the payment data

```
def successful_payment_mean(g):
    return g.loc[g['outcome'] == 'SUCCESS', 'amount'].mean()

payments.groupby('channel').apply(successful_payment_mean).reset_index(name='
average_successful_payment')
# APP    1781.861566
# WEB    1774.699689
```

Only SUCCESS rows count, because failed attempts also carry an amount and would distort the mean. The two channels collect nearly the same amount per successful payment. A function that sums one column and counts another follows in Part 12.

The warning and its fix

The notebook shows a DeprecationWarning: `DataFrameGroupBy.apply operated on the grouping columns`. The group passed to your function still contains the key column. There are two fixes, and both remove the key column from what `f` receives:

```
df.groupby('City')[['Sales', 'Cost']].apply(f)         # choose the columns
first
df.groupby('City').apply(f, include_groups=False)       # or drop the key
from the group
```

The functions above never use the key column, so their results are unaffected by the warning.

agg or apply?

	Named `agg`	`groupby().apply`
The function receives	one column, as a Series	the whole group, as a DataFrame
It returns	one number	one number, a Series or a DataFrame
Several result columns	many named results in one call	one function per call, joined afterwards
Speed	fastest with string names such as `'mean'`	slowest

Reach for `agg` first. Use `apply` when the calculation needs two columns or has to return a table.

Part 11. Group-aware row values

When every row needs a number computed from its own group, use `transform` rather than `apply`. The reason is that `apply` reorders the rows and drops the original index, while `transform` keeps both. The notebook hits this problem with `apply`, and this part shows the fix.

The task: each row's share of its group

```
df = pd.DataFrame({'City':  ['Delhi', 'Jaipur', 'Delhi', 'Jaipur'],
                   'Sales': [100, 200, 300, 600]})

df['Sales'] / df['Sales'].sum()           # share of overall sales
# 0.083333, 0.166667, 0.250000, 0.500000
```

For the share within a city, the notebook first did one city by hand and then used `groupby().apply`:

```
dff = df[df['City'] == 'Jaipur']
dff['Sales'] / dff['Sales'].sum()         # 0.25 and 0.75

def perc_sales_group(dff):
    return dff['Sales'] / dff['Sales'].sum()

df.groupby('City').apply(perc_sales_group).reset_index().drop(columns='level_
1')
#      City  Sales
# 0   Delhi   0.25
# 1   Delhi   0.75
# 2  Jaipur   0.25
# 3  Jaipur   0.75
```

The values are right, but the rows come back sorted by city and the original row labels are gone. They sat in the `level_1` column, which the notebook dropped. In the original order the shares are 0.25, 0.25, 0.75, 0.75. Assign the `apply` result by position and rows 1 and 2 receive the wrong values.

The same trap on the event data

```
def flag_events(dff):
    return dff['event_cost'] > dff['event_cost'].mean()

df_events['flag_events'] =
df_events.groupby('event_type').apply(flag_events).reset_index(drop=True)
```

The intent is to flag events that cost more than the average of their event type. The result comes back grouped by event type in alphabetical order, but `df_events` is in time order with the types mixed together. `reset_index(drop=True)` then pastes the flags in by

position, so they land on the wrong rows wherever the two orders differ. The notebook used the same shape for the share of channel cost (`calculate_percentage`), though it only displayed that result and did not assign it. Recompute the flag with `transform` (below) to check it.

The fix: transform

```
df['sales_share_in_city'] = df['Sales'] / df.groupby('City')
['Sales'].transform('sum')
# 0.25, 0.25, 0.75, 0.75      in the original row order

df_events['avg_cost'] = df_events.groupby('event_type')
['event_cost'].transform('mean')
df_events['above_average_cost'] = df_events['event_cost'] >
df_events['avg_cost']

df_events['channel_cost_share_pct'] = (
    df_events['event_cost'] / df_events.groupby('channel')
['event_cost'].transform('sum') * 100)
```

`transform('sum')` returns one value per row, equal to that row's group total, and it keeps the original index. Ordinary column arithmetic then does the rest. These are the cleaner answers to the last two exercises in the notebook: flag events above their type's average cost, and give each event's share of its channel's cost.

If you still want `apply`, use `df.groupby('City', group_keys=False).apply(f)`. It keeps the original row labels, so assigning the result aligns by label instead of by position.

Stratified sampling

```
dff = df_events[df_events['channel'] == 'APP']
dff.sample(frac=0.25)                       # 17,666 of the 70,662 APP rows

def sample_stratified(dff):
    return dff.sample(frac=0.25)

df_events.groupby('channel').apply(sample_stratified).reset_index(drop=True)
# 38,015 rows
```

Each channel contributes 25% of its own rows, so a small channel is not lost in the sample. The 38,015 rows are about a quarter of the 152,057. Here `reset_index(drop=True)` is harmless, because the sample is a new table and nothing is assigned back to the original. The built-in shortcut is `df_events.groupby('channel').sample(frac=0.25, random_state=42)`. Without a `random_state`, the sample changes on every run.

Part 12. Worked case studies on the payments data

On per-success cost, the APP channel is about 5.5% dearer than WEB (Rs 13.29 against Rs 12.60), even though its cost per attempt is lower. These cases combine the earlier patterns on the member-events data. Every number comes from your notebook outputs unless it is marked derived.

Case 1: which payment channel costs more per successful renewal?

The brief from the notebook: Finance wants the payment-processing cost per successful renewal for each channel, counting failed attempts too. Which channel looks more expensive, and why can an average cost per attempt mislead?

```
payments = df_events[df_events['event_type'] == 'PAYMENT_ATTEMPT']

payments['event_cost'].sum()                  # 116856  cost of every payment
attempt
(payments['outcome'] == 'SUCCESS').sum()      # 9024    successful attempts
116856 / 9024                                 # 12.95   per success, both
channels together

def get_cost_per_success(dff):
    total_cost = dff['event_cost'].sum()                 # failed attempts
are included
    success = (dff['outcome'] == 'SUCCESS').sum()
    if success > 0:
        return total_cost / success
    return np.nan

payments.groupby('channel').apply(get_cost_per_success).reset_index(name='cos
t_per_success')
#   channel  cost_per_success
# 0     APP         13.292791
# 1     WEB         12.604620
```

This needs `groupby().apply` because it uses two columns, `event_cost` and `outcome`. Failed attempts stay in the cost total and out of the success count, so their cost lands on the successes.

A per-attempt average misleads because payment rows carry only two costs. The failed rows shown in the notebook cost Rs 2, the successful ones cost Rs 12, and the summary shows a minimum of 2 and a maximum of 12. APP fails 39.26% of the time against 23.21% for WEB, so APP has more cheap failed rows pulling its per-attempt average down. From the notebook's counts the average per attempt works out to about Rs 8.07 for APP and Rs 9.68 for WEB. WEB looks dearer per attempt while APP is dearer per success. An APP success needs more attempts to get, and the per-attempt figure hides that. The two per-attempt averages are derived, and `payments.groupby('channel')['event_cost'].mean()` confirms them.

Case 2: failures, retries and cost side by side

Channel	Payment attempts	Renewals attempted	Renewals with 2+ attempts	Retry rate (derived)	Failure rate	Cost per success	Average successful amount
APP	7,445	5,822	1,183	20.3%	39.26%	Rs 13.29	Rs 1,781.86
WEB	5,863	5,296	564	10.7%	23.21%	Rs 12.60	Rs 1,774.70

APP fails more often, retries about twice as often and costs more per success. Both channels collect nearly the same amount per successful payment, so the gap is about reliability and not about price.

Case 3: members and payments by year

Event year	Events	Members with any event	Members with a successful payment	Share with a successful payment (derived)
2024	32,297	2,500	2,000	80.0%
2025	51,323	4,000	3,200	80.0%
2026	68,437	5,080	3,824	75.3%

The first three number columns come from Parts 7 and 8. The events begin on 2024-07-03 and end on 2026-10-09, so 2024 and 2026 are partial years and the 2026 share may not be final.

The recipe behind all three

1. Filter to the rows the question is about, such as `event_type == 'PAYMENT_ATTEMPT'`.
2. Group by the key you want to compare, such as `channel`.
3. Compute each measure with `agg`, a custom function or `apply`.
4. Reconcile: the per-channel pieces must add back to the overall figures, 116,856 for cost and 9,024 for successes.

Part 13. Pattern catalogue

Every exercise in the notebooks falls into one of the patterns below. Find the wording of your question in the left column to pick the tool, then follow the part number back to the worked example.

Question to pattern

The question says	Pattern	Skeleton	Part
rows or columns by name or by position	loc, iloc	`df.loc[labels, cols]` and `df.iloc[positions, cols]`	4
only the rows where	boolean mask with loc	`df.loc[df['x'] > 5, cols]`	5
more than one condition	& and parentheses	`df.loc[(a) & (b), cols]`	2, 5
change values where	loc assignment	`df.loc[mask, 'col'] = value`	5
label each row High or Low	Series.apply, or an if/elif function	`s.apply(make_bucket)`	3, 5

The question says	Pattern	Skeleton	Part
the result depends on several columns of one row	apply with axis=1	`df.apply(f, axis=1)`	5
fill gaps from another Series	masked assignment or fillna	`s[s.isna()] = other`	3
year or month from a date	.dt accessor	`s.dt.year`, `s.dt.strftime('%Y')`	6
how many rows per group	count	`agg(n=('id', 'count'))`	7
how many different X per group	nunique	`agg(n=('x', 'nunique'))`	7
total, average, earliest or latest per group	built-in function names	`agg(v=('x', 'mean'))`	7, 8
several measures per group	one agg call, many named results	`agg(a=(c1, 'count'), b=(c2, 'mean'))`	7
a measure for only some rows	filter first, then group	`df.loc[mask].groupby(k).agg(...)`	8
only the groups that pass a threshold	filter the aggregated table	`.loc[lambda d: d['n'] >= 300]`	8
is this key really unique	group by the key, keep counts above 1	`groupby(keys).agg(n=(c, 'count'))`	7
share of rows that meet a condition	custom function on a Series	`(s == 'FAILED').mean() * 100`	9
spread of a column	custom function	`s.max() - s.min()`	9

The question says	Pattern	Skeleton	Part
average of a subset, safe when empty	custom function with a length check	`s = s[s > 30]` then `s.mean()` if `len(s) > 0`	9
how many IDs repeat	value_counts	`(s.value_counts() > 1).sum()`	9
needs two columns per group	groupby.apply	`g.loc[g['a'] > g['b'], 'a'].mean()`	10
a value on every row, taken from its group	transform	`df.groupby(k)[c].transform('sum')`	11
share of the group total	column divided by a transform sum	`df[c] / df.groupby(k)[c].transform('sum')`	11
above the group average	column compared with a transform mean	`df[c] > df.groupby(k)[c].transform('mean')`	11
a sample from every group	groupby.sample	`df.groupby(k).sample(frac=0.25, random_state=42)`	11

Which function tool?

Tool	The function receives	It returns	Length of the result
`Series.apply(f)`	one value	one value	same as the Series
`DataFrame.apply(f, axis=1)`	one row, as a Series	one value	one per row
`groupby().agg(name=(col, f))`	one group's column	one number	one row per group
`groupby().apply(f)`	the whole group, as a DataFrame	a number, a Series or a DataFrame	varies, and the index may change

Tool	The function receives	It returns	Length of the result
`groupby()[col].transform(f)`	one group's column	values as long as the group	same as the frame, in the original order

Three habits behind every pattern

1. Work out one case by hand first, whether it is one value, one row or one group, and only then wrap it in a function.
2. Say what one row means, whether an event, a member or a renewal, and count the right thing: `count` for rows, `nunique` for distinct items.
3. Check the result against a total you already know or a group you computed by hand.

Part 14. Gotchas and bugs found in the notebooks

Two of these traps changed actual results in your notebooks: a cell that compounded a price rise, and an assignment that silently did nothing. A third, an apply result pasted back by position, very likely did too, and Part 11 shows how to check it. The table lists every trap with its cause and fix.

Symptom	Cause	Fix	Part
`KeyError: ('Sales', 'Product')`	a tuple is read as one column name	double brackets: `df[['Sales', 'Product']]`	4
FutureWarning on `ds[1]`	an integer key on a labelled Series is ambiguous	`ds.iloc[1]` for position	2
ValueError: identically-labeled Series	comparisons need identical indexes	align or reset the indexes first	3
NaN in a sum or product	a label exists on only one side	check the labels, or align before the arithmetic	3
a fill leaves a NaN (label 7 in the last fill exercise)	assignment aligns on labels and the other Series has no label 7	check that the filling Series has every label you need	3

Symptom	Cause	Fix	Part
`a <= 5 & b >= 2` raises an error	`&` binds tighter than the comparisons	parentheses around each comparison	2, 5
`loc[7:11]` returns 5 rows and `iloc[7:11]` returns 4	loc includes the end, iloc excludes it	pick the form that matches label or position	2, 4
wrong columns from a loc and iloc pair	`loc[7:11, ['Name', 'Age']]` and `iloc[7:12, 0:2]` are different selections	decide by label or by position and write one line	4
row 7 is not label 7	positions start at 0, so the 7th row has label 6 on a default index	check whether the wording says index or position	4
SettingWithCopyWarning and no change	chained brackets write into a temporary copy, or `dff = df.head()` is a derived slice	`df.loc[mask, 'col'] = value`, and `.copy()` for a slice you edit	5
FutureWarning: incompatible dtype	floats written into an int64 column	`astype(float)` first	5
Agent prices rose 21%, not 10%	the update cell was run twice	rebuild the frame, or write to a new column	5
NameError: name 'df' is not defined	the kernel restarted and the cell that creates `df` did not run	run the notebook from the top	5
NaN from an average	the mean of an empty Series is NaN	guard with `if len(s) > 0`	9
counts look too big or too small	`count` counts rows and `nunique` counts distinct items	choose by what the question asks	7
apply result lands on the wrong rows	`groupby().apply` returns group order and `reset_index(drop=True)` discards the labels	`transform`, or `group_keys=False`	11
DeprecationWarning on groupby.apply	the group still contains the key column	select columns first, or `include_groups=False`	10

Symptom	Cause	Fix	Part
yearly totals not comparable	events run from 2024-07-03 to 2026-10-09	group by fiscal period or month	7
a sample changes on every run	no `random_state`	`random_state=42`	11

What the double run did

In the travel notebook the Agent increase was run twice, so Dubai's base price went from 42,000 to 46,200 to 50,820, a 21% rise. Dubai's `total_price` shows 813,120 where a single 10% rise would give 739,200. Jaipur's shows 108,900 where it would give 99,000. The bucket labels for those two bookings are unchanged, but `new_price` and any total built on them are inflated.

What the chained assignment did

After `df.loc[df['Score'] < 50, 'Score'] = df['Score'] + 5` and then the chained version, Person_4 showed 6.326496, which is one rise of 5 from 1.326496. The second line changed nothing, and the warning was the only sign.

Part 15. Practice sets with worked solutions

The 20 Series questions from the first notebook are answered below, and the map after this paragraph points to where every other exercise is solved. Each answer lists its expected result as a comment, so you can check your own attempt.

Where the other exercises are solved

Exercise	Solved in
Ratings normalisation	Part 3
Rows 7, 9, 11 with City and Score, and with the last two columns	Part 4
Names from New York, score below the mean, age below the median	Part 5
Score under 50 plus 5	Part 5

Exercise	Solved in
Agent price rise, total_price, SAVE10 discount, booking_bucket, refundable price change, free luggage	Part 5
Group by exercises 1, 2, 3 and 7	Part 8
cost_range, discount_percentage, failure_percentage, average_high_cost, retry rate	Part 9
Profitable-sales average, high-experience salary, successful_payment_mean	Part 10
Share of group total, above-average flag, channel share, stratified sample	Part 11
Payment cost per successful renewal	Part 12

The 20 Series questions

```
import numpy as np
import pandas as pd

# 1. Select 'C103' and 'C106' in this order
s = pd.Series([1250, 980, 1540, 760, 2100, 1320], index=['C101', 'C102',
'C103', 'C104', 'C105', 'C106'])
s[['C103', 'C106']]                                    # C103 1540, C106 1320

# 2. Select the 2nd, 4th and 6th elements by position
s = pd.Series([72, 88, 91, 67, 85, 90, 79], index=['S1', 'S2', 'S3', 'S4',
'S5', 'S6', 'S7'])
s.iloc[[1, 3, 5]]                                      # S2 88, S4 67, S6 90

# 3. Slice 'Tue' through 'Sat' inclusive, by label
s = pd.Series([31.2, 32.0, 33.5, 34.1, 32.8, 31.9, 30.7], index=['Mon',
'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun'])
s['Tue':'Sat']                                         # 32.0, 33.5, 34.1,
32.8, 31.9

# 4. Update only 'P_D' to 10
s = pd.Series([14, 9, 11, 7, 15], index=['P_A', 'P_B', 'P_C', 'P_D', 'P_E'])
s['P_D'] = 10                                          # P_D is 10, the
```

```
others are unchanged

# 5. New Series with only r5, r2, r3 in this order
s = pd.Series([4.2, 3.8, 4.6, 2.9, 4.1], index=['r1', 'r2', 'r3', 'r4',
'r5'])
s[['r5', 'r2', 'r3']]                                  # r5 4.1, r2 3.8, r3
4.6

# 6. Values strictly greater than 25000
s = pd.Series([18000, 32000, 25000, 41000, 15000, 28000], index=['U1', 'U2',
'U3', 'U4', 'U5', 'U6'])
s[s > 25000]                                           # U2 32000, U4 41000,
U6 28000

# 7. Keep (s >= 25) and (s <= 35)
s = pd.Series([22, 35, 29, 41, 27, 33, 19], index=list('ABCDEFG'))
s[(s >= 25) & (s <= 35)]                               # B 35, C 29, E 27, F
33

# 8. Keep (s == 0) or (s >= 15)
s = pd.Series([0, 5, 12, 0, 31, 8, 0, 18], index=[f'L{i}' for i in range(1,
9)])
s[(s == 0) | (s >= 15)]                                # L1 0, L4 0, L5 31,
L7 0, L8 18

# 9. Keep entries that are not missing and above 900
s = pd.Series([1200, np.nan, 450, 2100, np.nan, 980, 1550],
index=list('abcdefg'))
s[(s.notna()) & (s > 900)]                             # a 1200, d 2100, f
980, g 1550

# 10. New Series where values equal to 0 or 1 become -1
s = pd.Series([3, 0, 5, 2, 0, 1, 4, 2], index=['K1', 'K2', 'K3', 'K4', 'K5',
'K6', 'K7', 'K8'])
s.mask((s == 0) | (s == 1), -1)                        # 3, -1, 5, 2, -1, -1,
4, 2

# 11. Mean of the values from 120 to 260 inclusive
s = pd.Series([180, 240, 95, 310, 130, 260, 150, 400], index=[f'req{i}' for i
in range(1, 9)])
s[(s >= 120) & (s <= 260)].mean()                      # 192.0  (180, 240,
130, 260, 150)
```

```
# 12. Miles to km, rounded to 2 decimals, with apply
s = pd.Series([1.2, 3.5, 0.8, 10.1, 6.6], index=['t1', 't2', 't3', 't4',
't5'])
s.apply(lambda x: round(x * 1.60934, 2))              # 1.93, 5.63, 1.29,
16.25, 10.62

# 13. LOW below 0.4, MID from 0.4 to 0.7 inclusive, otherwise HIGH
s = pd.Series([0.12, 0.66, 0.48, 0.91, 0.35, 0.70, 0.05], index=[f'p{i}' for
i in range(1, 8)])
def band(x):
    if x < 0.4:
        return 'LOW'
    elif x <= 0.7:
        return 'MID'
    else:
        return 'HIGH'
s.apply(band)                                         # LOW, MID, MID, HIGH,
LOW, MID, LOW

# 14. Strip the rupee sign and commas, convert to int
s = pd.Series(['₹12,500', '₹999', '₹2,050', '₹18,000', '₹7,499'], index=
['i1', 'i2', 'i3', 'i4', 'i5'])
s.apply(lambda x: int(x.replace('₹', '').replace(',', '')))    # 12500, 999,
2050, 18000, 7499

# 15. Strip spaces and convert to title case
s = pd.Series(['  ankit  ', 'Riya', '  MOHAN', 'neha  ', '  ViKRaM '], index=
['n1', 'n2', 'n3', 'n4', 'n5'])
s.apply(lambda x: x.strip().title())                  # Ankit, Riya, Mohan,
Neha, Vikram

# 16. Keep 'delhi' or 'pune'
s = pd.Series(['delhi', 'mumbai', 'pune', 'delhi', 'jaipur', 'pune',
'delhi'], index=[f'c{i}' for i in range(1, 8)])
s[(s == 'delhi') | (s == 'pune')]                     # c1, c3, c4, c6, c7

# 17. Keep (s <= 0) or (s >= 500)
s = pd.Series([500, -120, 0, 240, -5, 910, 30], index=['A1', 'A2', 'A3',
'A4', 'A5', 'A6', 'A7'])
s[(s <= 0) | (s >= 500)]                              # A1 500, A2 -120, A3
0, A5 -5, A6 910

# 18. Starts with 'AX' and longer than 4 characters
```

```
s = pd.Series(['AX-120', 'BX-9', 'AX-450', 'CX-12', 'AX-7', 'BX-200'], index=
['s1', 's2', 's3', 's4', 's5', 's6'])
s[(s.str.startswith('AX')) & (s.str.len() > 4)]        # s1 AX-120, s3 AX-450
('AX-7' has length 4)

# 19. revenue = qty * price, then keep 300 to 700 inclusive
idx = ['o1', 'o2', 'o3', 'o4', 'o5', 'o6', 'o7']
qty = pd.Series([2, 5, 1, 8, 3, 10, 6], index=idx)
price = pd.Series([120, 80, 200, 50, 150, 40, 90], index=idx)
revenue = qty * price                                  # 240, 400, 200, 400,
450, 400, 540
revenue[(revenue >= 300) & (revenue <= 700)]           # o2, o4, o5, o6, o7

# 20. (a) keep not missing and above 0.2   (b) missing becomes -1, with apply
s = pd.Series([0.10, 0.55, np.nan, 0.92, 0.30, 0.75, np.nan, 0.05], index=
[f'u{i}' for i in range(1, 9)])
s[(s.notna()) & (s > 0.2)]                             # u2 0.55, u4 0.92, u5
0.30, u6 0.75
s.apply(lambda x: -1 if pd.isna(x) else x)             # u3 and u7 become -1,
same as s.fillna(-1)
```

Question 18 uses the `.str` accessor, which runs a string method on every value of a Series. Questions 14 and 15 do similar string work through `apply`.

What each question trains

Questions	Skill	Part
1 and 5	pick by a list of labels, in a chosen order	2
2	pick by position with `iloc`	2
3	label slice, end included	2
4	assign to one label	2
6	one condition as a mask	2
7, 8, 9, 16, 17, 18 and 20a	two conditions joined with the and/or operators	2
10	replace values where a mask holds	5
11	aggregate a filtered Series	2, 7
12, 13, 14, 15 and 20b	`Series.apply` with a function or a lambda	3

Questions	Skill	Part
19	arithmetic aligned on the index, then a mask	2, 3

Unfinished in the notebooks

Each item below has a worked answer in the part named.

- Part 4: rows 7, 9 and 11 with City and Score, and with the last two columns.
- Part 5: the SAVE10 discount and the free luggage allowance.
- Part 8: Exercise 1 (the cell stopped mid-line), Exercise 2 (not run), the 300-member filter in Exercise 3, and the two date columns in Exercise 7.
- Part 9: `event_count` in the cost-range exercise, `total_events` in the high-cost exercise, and the retry-rate division.
- Part 11: the above-average flag and the channel cost share, redone with `transform`.